Arquitecturas y lenguajes de programación en clientes web

Caso práctico



Ministerio de Educación (Uso educativo nc)

La empresa "**BK Programación**", ha sido informada a través de uno de sus asesores, de que se ha abierto el plazo, para concursar a la adjudicación de un proyecto de modernización de la web de una importante empresa dedicada a la moda.

Ada, la directora de "**BK Programación**", decide que es una buena oportunidad para conseguir trabajo para su empresa, por lo que consultan el pliego de requisitos exigidos y ve que su empresa tiene personal suficiente para dar solución a dicho proyecto. Se envía la solicitud y dos meses más tarde, la empresa de **Ada** sale adjudicataria del contrato de modernización.

El objetivo principal del proyecto es el de dar un mayor dinamismo a las páginas y actualizar la web a lo que se conoce como Web 2.0 (mayor interacción, interoperabilidad, aplicaciones más ricas y no intrusivas, etc.)

Ada considera que **Antonio** (estudiante de ciclo y trabajador de su empresa), con conocimientos de lenguaje HTML, podría formarse en las técnicas necesarias para realizar dicho trabajo de actualización y como además el proyecto tendrá un plazo máximo de entrega de 1 año, dispondrá de tiempo más que suficiente para ello.

Ada, informa a sus trabajadores de la adjudicación de dicho contrato, y **Antonio** (bajo la tutoría de **Juan**), decide comenzar a investigar en las diferentes opciones disponibles para llevar a cabo su trabajo.

En esta unidad de trabajo nos vamos a introducir en los tipos de lenguajes de diseño web en entorno cliente, sus características, sus limitaciones, aspectos de seguridad y veremos algunas herramientas que podrás utilizar para configurar tu entorno de trabajo y comenzar a programar en JavaScript.

¡Pues vayamos a ello!



Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Desarrollo web

Caso práctico



Ministerio de Educación (Uso educativo nc)

Juan (técnico en Desarrollo de Aplicaciones Informáticas), será el tutor de **Antonio** en la empresa **BK Programación** y le irá guiando durante todo el proceso de formación y en la realización del proyecto de modernización de la web de la compañía de moda.

Para comenzar, **Juan** le recomienda a **Antonio** una pequeña introducción sobre lo que es el Desarrollo Web y las diferentes áreas que abarca el Diseño Web.



Silvio Tanaka (CC BY-NC)

La web fue inicialmente concebida y creada por **Tim Berners-Lee**, un especialista del laboratorio europeo de partículas (CERN) en 1989. En sus mismas palabras, había una "necesidad de una herramienta colaborativa que soportara el conocimiento científico" en un contexto internacional. Él y su compañero **Robert Cailliau** crearon un prototipo web para el CERN y lo mostraron a la comunidad para sus pruebas y comentarios.

Dicho prototipo estaba basado en el concepto de hipertexto. Como resultado se crearon unos protocolos y especificaciones que han sido adoptados universalmente e incorporados a Internet, gracias a aportaciones posteriores como el desarrollo por la NCSA de la popular interfaz MOSAIC.

Todos los prototipos y desarrollos posteriores crecieron bajo la guía del **consorcio W3C**, que es una organización con base en el MIT de Massachusetts y que se responsabiliza de desarrollar y mantener los estándares web.

Por **Web se pueden entender tres cosas distintas:**

- El proyecto inicial del CERN
- El conjunto de protocolos desarrollados en dicho proyecto
- El espacio de información formado por todos los servidores interconectados. Cuando se habla de la Web generalmente se hace referencia a esto último.

Muchas de las discusiones sobre Diseño Web o Desarrollo Web son confusas, ya que la expresión varía considerablemente. Mientras que la mayoría de la gente tiene algún tipo de noción sobre **lo que significa Diseño Web**, solamente unos pocos son capaces de definirlo con exactitud, y tú vas a estar dentro de ese grupo.

Algunos componentes como diseño gráfico o programación forman parte de esa discusión, pero su importancia en la construcción de webs varía de persona a persona y de web a web. Algunos consideran la creación y organización de contenido - o más formalmente, la arquitectura de la información - como el aspecto más importante del Diseño Web. Otros factores como, la facilidad de uso, el valor y funcionalidad del sitio web en la organización, su funcionalidad, accesibilidad, publicidad, etc. también forman una parte muy activa hoy en día sobre lo que se considera Diseño Web.

El Desarrollo Web ha sido y sigue estando muy influenciado por múltiples campos como el de las nuevas tecnologías, los avances científicos, el diseño gráfico, la programación, las redes, el diseño de interfaces de usuario, la usabilidad y una variedad de múltiples recursos. Por lo tanto, el Desarrollo Web es realmente un campo multidisciplinar.

Autoevaluación

Cuando nos referimos a la Web ¿estamos hablando de?

- ○ Proyecto del CERN.
- ○ Protocolos utilizados en el proyecto del CERN.
- ○ Espacio de información que nos proporcionan los servidores a través de Internet.

No es correcta, ya que dicho proyecto es el origen de lo que hoy entendemos por la Web.

No es cierto, ya que los protocolos simplemente son un componente más que se utiliza para el buen funcionamiento de las comunicaciones en la Web.

Efectivamente es correcta, ya que hoy en día cuando hablamos de Web estamos hablando de todo ese entramado de información proporcionado por los servidores de información interconectados.

Solución

1. Incorrecto

2. Incorrecto
3. Opción correcta

1.1.- Conceptos básicos

Antes de entrar de lleno en el desarrollo web es esencial definir los conceptos básicos con los que trabajaremos:

- **Página web.** Es un documento único escrito en el lenguaje de etiquetas HTML. En este lenguaje se define su estructura, mientras que su estilo es posteriormente mediante el lenguaje CSS y su interactividad, con JavaScript.
- **Sitio web.** Es el conjunto de páginas web relacionadas entre sí y accesibles a través de un nombre de dominio único.
- **Aplicación web.** Es un tipo de aplicación que se ejecuta en una página web. Las aplicaciones son más completas que una página web normal, requiriendo una mayor interactividad. Un ejemplo, de aplicación web es Google Docs.

Para saber más

Los gestores de contenido son herramientas para la creación de webs. Estos se caracterizan por ser fáciles de utilizar y personalizar, requiriendo poco conocimiento técnico. Algunos de los gestores de contenido más conocidos son:

- WordPress. Permite la creación de cualquier página web, es utilizado habitualmente para la creación de blogs, aunque también puede utilizarse con otros propósitos (por ejemplo, el comercio online).
- Moodle. Permite crear aulas virtuales con propósito educativo. Por ejemplo, el aula virtual de este curso está creada utilizando esta tecnología.
- PrestaShop. Enfocada en el comercio online.

Todas ellas cuentan con diferentes plantillas que permiten personalizar su aspecto visual, así como *plugins* para extender sus funcionalidades.

1.2.- Áreas

Hay cinco áreas que cubren la mayor parte de las facetas del Diseño Web:



200degrees (Dominio Público)

- **Contenido**: incluye la forma y organización del contenido del sitio. Esto puede abarcar desde cómo se escribe el texto hasta cómo está organizado, presentado y estructurado usando tecnologías de marcas como HTML.
- **Visual**: hace referencia a la plantilla empleada en un sitio web. Esta plantilla generalmente se genera usando CSS y puede incluir elementos gráficos para decoración o para navegación. El aspecto visual es el aspecto más obvio del Diseño Web, pero no es la única disciplina o la más importante.
- **Tecnología**: aunque muchas de las tecnologías web como HTML o CSS entran dentro de esta categoría, la tecnología en este contexto generalmente hace referencia a los diferentes tipos de elementos interactivos de un sitio web, generalmente aquellos construidos empleando técnicas de programación.
- **Distribución**: la velocidad y fiabilidad con la que un sitio web se distribuye en Internet o en una red interna corporativa está relacionado con el hardware/software utilizado y el tipo de arquitectura de red utilizada en la conexión.
- **Propósito**: la razón por la que un sitio web existe generalmente está relacionada con algún aspecto de tipo económico. Por lo tanto, este elemento debería considerarse en todas las decisiones que tomemos en las diferentes áreas.

El porcentaje de influencia de cada una de estas áreas en un sitio web, puede variar dependiendo del tipo de sitio que se está construyendo. Una página personal generalmente no tiene las consideraciones económicas que tendría una web que va a vender productos en Internet.

1.3.- Front-end y back-end

Las tecnologías de desarrollo web se clasifican en lado cliente (front-end) y lado servidor (back-end), unidos a través de la red que permite la conectividad del sitio web con el usuario.



Sara_Torda (Dominio Público)

- **Cliente (front-end)**: Las tecnologías front-end son aquellas que son ejecutadas en el cliente, generalmente en el contexto del navegador web. Podemos distinguir dos variantes:
  - Lenguajes que nos permiten dar formato y estilo a una página web (HTML, CSS, etc.).
  - Lenguajes que nos permite aportar dinamismo a páginas web (lenguajes de scripting, tales como javascript o typescript).

- **Servidor (back-end)**: Las tecnologías back-end son aquellas que son ejecutadas o interpretadas por el servidor. Incluye el hardware y software del servidor Web así como diferentes elementos de programación y tecnologías incrustadas. Las tecnologías pueden abarcar un rango amplio desde programas CGI escritos en PERL hasta aplicaciones multihilo basadas en Java, incluyendo tecnologías de servidor de bases de datos que soporten múltiples sitios web. Hoy día se utiliza mucho en el lado servidor el lenguaje **node.js** basado en JavaScript, pero para el lado del servidor.

La realidad es que cada tecnología tiene sus pros y sus contras. En general, las tecnologías front-end y back-end poseen características que las hacen complementarias más que adversarias. Por ejemplo, cuando añadimos un formulario para recoger información y grabarla en una base de datos, es obvio que tendría más sentido chequear el formulario en el lado del cliente para asegurarnos que la información introducida es correcta, justo antes de enviar la información a la base de datos del servidor, aunque siempre también deberemos validar dicha información en el lado del servidor para asegurar la integridad. La programación en el lado del cliente consigue que la validación del formulario sea mucho más efectiva y que el usuario se sienta menos frustrado al cubrir los datos en el formulario. Por otro lado, el almacenar los datos en el servidor estaría mucho mejor gestionado por una tecnología del lado del servidor (server-side), dando por supuesto que la base de datos estará en el lado del servidor.

Tabla comparativa de lenguajes de programación web cliente – servidor

Lado del Cliente (client-side)	Lado del servidor (server-side)
Lenguajes de formato y estilo: HTML y HTML5. CSS y CSS3. SASS. Frameworks de estilo: Bootstrap. Tailwind CSS. Lenguajes de scripting:	Scripts y programas CGI. Programas API del servidor. Módulos de Apache. Extensiones ISAPI y filtros. Servlets de Java. Lenguajes de scripting. PHP. Laravel (framework de PHP).

Lado del Cliente (client-side)	Lado del servidor (server-side)
- JavaScript. - VBScript. - TypeScript. - Frameworks de scripting: - React. - Angular. - Vue.js. - Next.js	- Active Server Pages (ASP/ASP.NET). - Node.js (JavaScript). - Flask (Python).

Debes conocer

- Ruta de aprendizaje del desarrollador *front-end*: https://roadmap.sh/frontend
  - Puedes profundizar en algunos de estos conceptos en la formación de Mozilla Developer: https://developer.mozilla.org/es/docs/orphaned/Learn/Front-end_web_developer
- Ruta de aprendizaje del desarrollador *back-end*: https://roadmap.sh/backend

Para saber más

La creación visual de una página web es tarea del diseñador web, quien se encarga de establecer la identidad visual y la jerarquía de la misma. Este proceso se concreta en la elaboración de una guía de estilo (aquí puedes ver la guía de estilo de la web de la Universidad de Córdoba). Para diseñar esta guía, el diseñador web utiliza herramientas especializadas como como figma o whimsical. Una vez completada, la guía de estilo es implementada por un desarrollador web front-end, quien se encarga de traducir estos elementos visuales y directrices en código funcional que dará vida a la página web.

Autoevaluación

La programación en el lado del servidor se encarga de la presentación visual y estilo de la página web.

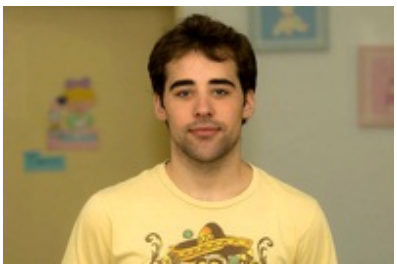
○ Verdadero ○ Falso

Falso

Falso. La programación en el lado del servidor se encarga de ejecutar programas y scripts que interactúan con bases de datos y aplicaciones del servidor, no de la presentación visual y estilo de la página web.

2.- Lenguajes de programación en clientes web

Caso práctico



Ministerio de Educación (Uso educativo nc)

El proyecto de modernización de la empresa de moda va a consistir, principalmente, en añadir mayor funcionalidad a las páginas ya existentes, más control en los formularios, dinamismo, modernización de la mecánica de varios procesos, etc.

Antonio tendrá que centrarse en los lenguajes de programación en el lado del cliente, ya que éstos son los que le ayudarán a aportar las capacidades solicitadas en el proyecto.

Antonio se pone manos a la obra y consulta bajo la tutoría de **Juan**, qué opciones tienen para llevar a cabo esa programación y analizan las características, compatibilidades y seguridad aportada por los lenguajes de programación en el lado del cliente.

Uno de los objetivos en la programación web es saber **escoger la tecnología correcta** para tu trabajo. Muchas veces los desarrolladores escogen las tecnologías que le son más familiares, generalmente JavaScript (front-end) y PHP (back-end), y la usan en todas las situaciones. Sin embargo, cada tecnología ofrece sus ventajas y desventajas y, por tanto, es fundamental elegir la que más se adapte a nuestras necesidades. Además, la elección de una tecnología de front-end puede limitar que tecnología de back-end podemos utilizar, y viceversa. Por ejemplo, si en el front-end elegimos utilizar un framework, como React o Angular, las peticiones al servidor tendrán que realizarse mediante AJAX, lo cual requiere que el servidor despliegue una API Rest para permitir las consultas.

En el lado front-end, las diferentes tecnologías que componen una web se dividen en cuatro capas (aquí puedes ver un ejemplo):

Tabla comparativa de lenguajes de programación web cliente – servidor.

Capa	Propósito	Lenguajes
Contenido	El texto, imágenes y demás contenido multimedia que compone nuestra web.	-

Capa	Propósito	Lenguajes
Estructura	Como se organizan las diferentes partes de una web, así como su significado semántico (cabecera, pie de página, parte principal, etc.).	HTML, HTML5
Presentación	Indica como se debe visualizar cada componente del archivo .html, desde colores o tipografía a alineación y distribución de los elementos. Esta parte es fundamental para hacer nuestra página responsive, es decir, que se visualice adecuadamente en cualquier dispositivo.	CSS, SASS,
Comportamiento	Esta última capa es la que proporciona el comportamiento de la página web, permitiendo que esta reaccione a las acciones del usuario.	JavaScript, TypeScript

En este módulo nos vamos a centrar en la capa de comportamiento y, en particular, en el lenguaje JavaScript. Javascript es el lenguaje de script más utilizado en la programación en el lado del cliente, y está soportado mayoritariamente por todas las plataformas. Además, es la base para el resto puesto que por ejemplo el código TypeScript (habitualmente llamado JavaScript con tipos) es compilado a código JavaScript. Del mismo modo, los componentes realizados en frameworks, como React, se realizan utilizando código JavaScript. Sin olvidar, Node.js que extiende la funcionalidad de JavaScript al lado del servidor.

2.1.- Características

Como vimos anteriormente, los lenguajes de programación para clientes web no son un reemplazo de la programación en el lado del servidor. Cualquier web que reaccione dinámicamente a interacciones del usuario o que almacene datos, estará gestionada por lenguajes de script en el lado del servidor, incluso aunque usemos JavaScript en el cliente para mejorar la experiencia de usuario. Las razones son simples:



Everaldo Coelho (GNU/GPL)

- **Primero**: JavaScript por sí mismo no puede escribir ficheros en el servidor. Puede ayudar al usuario a elegir opciones o preparar datos para su envío, pero después de eso solamente podrá ceder los datos al lenguaje de servidor encargado de la actualización de datos.
- **Segundo**: no todos los clientes web ejecutan JavaScript. Algunos lectores, dispositivos móviles, buscadores, o navegadores instalados en ciertos contextos están entre aquellos que no pueden realizar llamadas a JavaScript, o que simplemente son incompatibles con el código de JavaScript que reciben. Aunque ésto ocurra nuestra página web debería ser completamente funcional con JavaScript desactivado. Utilizaremos JavaScript para conseguir que la experiencia de navegación web sea lo más rápida, moderna o divertida posible, pero no dejaremos que nuestra web deje de funcionar si JavaScript no está funcionando.
- **Tercero**: uno de los caminos que más ha integrado la programación cliente con la programación servidor ha surgido gracias a **AJAX**. El proceso "asíncrono" de AJAX se ejecuta en el navegador del cliente y emplea JavaScript. Este proceso se encarga de solicitar datos XML, o enviar datos al lenguaje de servidor y todo ello de forma transparente en background. Los datos devueltos por el servidor pueden ser examinados por JavaScript en el lado del cliente, para actualizar secciones o partes de la página web. Es así como funcionan hoy día la mayoría de las webs.

JavaScript está orientado a dar soluciones a:

- Conseguir que nuestra página web responda o reaccione directamente a la interacción del usuario con elementos de formulario y enlaces hipertexto.
- La distribución de pequeños grupos de datos y proporcionar una interfaz amigable para esos datos.
- Controlar múltiples ventanas o marcos de navegación, plug-ins, o applets Java basados en las elecciones que ha hecho el usuario en el documento HTML.
- Pre-procesar datos en el cliente antes de enviarlos al servidor.
- Modificar estilos y contenido en los navegadores de forma dinámica e instantáneamente, en respuesta a interacciones del usuario.
- Solicitar ficheros del servidor, y enviar solicitudes de lectura y escritura a los lenguajes de servidor.

Los lenguajes de script como JavaScript no se usan solamente en las páginas web. Los intérpretes de JavaScript están integrados en múltiples aplicaciones de uso cotidiano. Estas aplicaciones proporcionan su propio modelo de acceso y gestión de los módulos que componen la aplicación y para ello comparten el lenguaje JavaScript en cada aplicación.

Autoevaluación

¿Podríamos escribir con JavaScript un fichero de texto directamente en el servidor?

- ◯ Sí, pero sólo si se trata de una cookie.
- ◯ De ninguna manera.
- ◯ Sí, empleando AJAX.

No es correcta, ya que aunque JavaScript puede escribir cookies lo hará en el lado del cliente.

Efectivamente es correcta ya que JavaScript es un lenguaje que interpreta el navegador web y no es un lenguaje de servidor, por lo que no tendrá acceso a los recursos para poder escribir en él.

No es cierto, ya que AJAX lo que nos permitirá es hacer peticiones en background al servidor pero no escribir nada directamente en el servidor puesto que AJAX está basado en JavaScript y es una tecnología en el lado del cliente.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

2.2.- Compatibilidades



Everaldo Coelho (GNU/GPL)

A diferencia de otros tipos de scripts, JavaScript es interpretado por el cliente. Actualmente existen múltiples clientes o navegadores que soportan JavaScript, incluyendo Firefox, Google Chrome, Safari, Opera, Internet Explorer, etc. Por lo tanto, cuando escribimos un script en nuestra página web, tenemos que estar seguros de que será interpretado por diferentes navegadores y que aporte la misma funcionalidad y características en cada uno de ellos. Ésta es otra de las diferencias con los scripts de servidor en los que nosotros dispondremos del control total sobre su interpretación.

Cada tipo de navegador da soporte a diferentes características del JavaScript y además también añaden sus propios **bugs o fallos**. Algunos de estos fallos son específicos de la plataforma sobre la que se ejecuta ese navegador, mientras que otros son específicos del propio navegador en sí.

Para saber más

¿Te gustaría ver cómo soporta cada navegador las diferentes características de Javascript y poder verlo por versiones?

Comparación de navegadores web.

Por otro lado, si te interesa alguna funcionalidad en concreto puedes utilizar la web de:

CanIUse

A veces las incompatibilidades entre navegadores al interpretar el código de JavaScript no vienen dadas por el propio código en sí, sino que su origen proviene del código fuente HTML. Por lo tanto, es muy importante que tu código HTML siga las **especificaciones del estándar W3C** y para ello dispones de herramientas como el validador HTML W3C:

Validador W3C.

También tienes que tener precaución con las **limitaciones en el uso de JavaScript**:

- No todos los navegadores soportan lenguajes de script (en especial JavaScript) en el lado del cliente.
- Algunos dispositivos móviles tampoco podrán ejecutar JavaScript.
- Incluso las implementaciones más importantes de JavaScript en los diferentes navegadores no son totalmente compatibles entre ellas: por ejemplo diferentes

incompatibilidades entre Firefox e Internet Explorer.

- La ejecución de código JavaScript en el cliente podría ser desactivada por el usuario de forma manual, con lo que no podremos tener una confianza ciega en que se vaya a ejecutar siempre tu código de JavaScript.
- Algunos navegadores de voz no interpretan el código de JavaScript.

Autoevaluación

JavaScript es un lenguaje de programación que se ejecuta exclusivamente en el navegador web y permite dar dinamismo a las webs.

○ Verdadero ○ Falso

Falso

Falso. Aunque JavaScript es un lenguaje de programación que se ejecuta comúnmente en el navegador web para hacer las páginas web dinámicas, no se limita únicamente al navegador. JavaScript también puede ejecutarse en el lado del servidor utilizando entornos como Node.js. Esto permite que JavaScript se use tanto para el desarrollo del front-end como para el back-end, haciendo que sea un lenguaje de programación muy versátil.

2.3.- Seguridad



Everaldo Coelho (GNU/GPL)

JavaScript proporciona un gran potencial para diseñadores maliciosos que quieran distribuir sus scripts a través de la web. Para evitar ésto los navegadores web en el cliente aplican dos tipos de restricciones:

- Por razones de seguridad cuando se ejecuta código de JavaScript éste lo hace en un "espacio seguro de ejecución" en el cuál solamente podrá realizar tareas relacionadas con la web, nada de tareas genéricas de programación como creación de ficheros, etc.
- Además, los scripts están restringidos por la política de "mismo origen": la cual quiere decir que los scripts de una web no tendrán acceso a información tal como usuarios, contraseñas, o cookies enviadas desde otra web. La mayor parte de los agujeros de seguridad son infracciones tanto de la **política** de "**mismo origen**" como de la política de "**espacio seguro de ejecución**".

Al mismo tiempo es importante entender las **limitaciones que tiene JavaScript** y que, en parte, refuerzan sus capacidades de seguridad. JavaScript no podrá realizar ninguna de las siguientes tareas:

- Modificar o acceder a las preferencias del navegador del cliente, las características de apariencia de la ventana principal de navegación, las capacidades de impresión, o a los botones de acciones del navegador.
- Lanzar la ejecución de una aplicación en el ordenador del cliente.
- Leer o escribir ficheros o directorios en el ordenador del cliente (con la excepción de las cookies).
- Escribir directamente ficheros en el servidor.
- Capturar los datos procedentes de una transmisión en streaming de un servidor, para su retransmisión.
- Enviar e-mails a nosotros mismos de forma invisible sobre los visitantes a nuestra página web (aunque si que podría enviar datos a una aplicación en el lado del servidor capaz de enviar correos).
- Interactuar directamente con los lenguajes de servidor.
- Las páginas web almacenadas en diferentes dominios no pueden ser accesibles por JavaScript.
- JavaScript es incapaz de proteger el origen de las imágenes de nuestra página.
- Implementar multiprocesamiento o multitarea.
- Otro tipo de **vulnerabilidades** que podemos encontrar están relacionadas con el XSS. Este tipo de vulnerabilidad viola la política de "**mismo origen**" y ocurre cuando un atacante es capaz de inyectar código malicioso en la página web presentada a su víctima. Este código malicioso puede provenir de la base de datos a la cual está accediendo esa víctima. Generalmente este tipo de errores se deben a fallos de implementación de los programadores de navegadores web.

Otro aspecto muy relacionado con la seguridad son los defectos o imperfecciones de los navegadores web o plugins utilizados. Estas imperfecciones pueden ser empleadas por los atacantes para escribir scripts maliciosos que se puedan ejecutar en el sistema operativo del usuario.

El **motor de ejecución de JavaScript** es el encargado de ejecutar el código de JavaScript en

el navegador y por lo tanto es en él dónde recaerá el peso fuerte de la implementación de la seguridad. Podríamos citar varios ejemplos de motores de JavaScript:

- Active Script de Microsoft: tecnología que soporta JScript como lenguaje de scripting. A menudo se considera compatible con JavaScript, pero Microsoft emplea múltiples características que no siguen los estándares ECMA.
- El kit de herramientas Qt desarrollado en C++ también incluye un módulo intérprete de JavaScript.
- El lenguaje de programación Java en su versión JDK 1.6 introduce un paquete denominado javax.script que permite la ejecución de JavaScript.
- Y por supuesto todos los motores implementados por los navegadores web como Mozilla, Google, Opera, Safari, etc. Cada uno de ellos da soporte a alguna de las diferentes versiones de JavaScript.

Hoy en día una de las características que más se resalta y que permite diferenciar a unos navegadores de otros, es la rapidez con la que sus motores de JavaScript pueden ejecutar las aplicaciones, y la seguridad y aislamiento que ofrecen en la ejecución de las aplicaciones en diferentes ventanas o pestañas de navegación.

Para saber más

Uno de los ataques que puede sufrir una web es el de inyección SQL. En estos, el atacante aprovecha nuestros formularios para realizar consultas indebidas a nuestra base de datos. La primera forma de afrontarlo es validar nuestros formularios, o cualquier otro punto a través del cual el usuario pueda introducir información, en el front-end. Sin embargo, comprobar los datos solo en el front-end no es suficiente, ya que el atacante puede fácilmente eludir esta protección. Por lo tanto, es necesario realizar de nuevo esta validación en el back-end.

También hay que tener cuidado con los archivos que permitimos subir a nuestro servidor. Por ejemplo, si el atacante sube el archivo C99 WebShell, podría llegar a tomar el control de nuestro servidor. Para evitar esto, es fundamental implementar controles estrictos sobre los tipos de archivos permitidos y realizar escaneos de seguridad en los archivos subidos.

2.4.- Frameworks, APIs y librerías de desarrollo front-end

La complejidad de algunas aplicaciones Web requieren del uso de *frameworks,* librerías y APIs diseñado para facilitar el trabajo de los desarrolladores.

Frameworks de desarrollo front-end.

Los *frameworks* más populares, según StackOverflow y StateOfJs, son React, Angular y Vue.js. Estos utilizan un enfoque basado en componentes que permite encapsular tanto el diseño como el comportamiento de un elemento (por ejemplo, un acordeón), facilitando su reutilización en diversas partes de una aplicación web. Además, ofrecen la posibilidad de emplear componentes de terceros.

Framework	Creador	Descripción	Ejemplos de uso
React	Facebook	React es un framework diseñado para construir interfaces de usuario con un enfoque en componentes reutilizables y modulares (por ejemplo, en materialUI pueden verse varios ejemplos de componentes realizados con React). Utiliza JSX para escribir componentes que encapsulan tanto su aspecto visual, con HTML y CSS, como su comportamiento, con JavaScript o TypeScript. Además, optimiza las interacciones y actualizaciones haciendo uso de un DOM virtual.	Facebook, Instagram, WhatsApp Web
Angular	Google	Angular ofrece una plataforma robusta para el desarrollo de aplicaciones web y móviles, siguiendo un enfoque basado en componentes y servicios, a la vez que incorpora elementos del patrón modelo-vista-controlador (MVC) para facilitar la gestión de la lógica de negocio, la interfaz de usuario y la manipulación de datos.	Gmail, Paypal, Youtube
Vue.js	Evan You	Vue.js es un framework ligero y versátil para el desarrollo de interfaces web, destacando por su facilidad de aprendizaje comparado con React y Angular. Es ideal para proyectos que requieren soluciones rápidas y eficientes.	Netflix, Gitlab, Laravel

Framework	Creador	Descripción	Ejemplos de uso
Laravel Livewire	Caleb Porzio	Livewire permite programar componentes front-end sencillos utilizando PHP. Sus funcionalidades pueden ser extendidas utilizando Blade o Filament. Nótese, que crear un back-end con Laravel no nos limita a elegir como front-end Laravelwire, sino que también se podría hacer uso de otros frameworks como React o Vue. Dependiendo en última instancia la decisión de las necesidades del proyecto.	Utilizado principalmente en proyectos internos y aplicaciones de tamaño mediano.

APIs y librerías de desarrollo.

JavaScript, como lenguaje de programación principal en el desarrollo web, tiene una rica colección de APIs (proporcionadas por el propio navegador web) y librerías (integradas por nosotros a nuestro proyecto) que permiten a los desarrolladores ampliar sus funcionalidades. Las más conocidas son:

- **Manipulación de la interfaz de usuario**
  - Intersection Observer API (API). Permite detectar cuando un elemento es visible en la página web. Es fundamental para realizar animaciones basadas en *scroll*.
  - MutationObserver (API). Permite detectar cuando un elemento de la página, o sus hijos, es modificado.
  - jQuery (Librería): Simplifica la manipulación del DOM, gestión de eventos y animaciones. Actualmente, con un uso cada vez menor debido al avance de JavaScript en los últimos años.
- **Edición de Texto Enriquecido**
  - TinyMCE (Librería): Provee un editor de texto WYSIWYG integrable en plataformas web.
- **Visualización de Datos y Gráficos**
  - Chart.js (Librería): Permite la creación de gráficos interactivos en el navegador.
  - D3.js (Librería): Librería potente para manipular documentos basados en datos, ideal para visualizaciones complejas y gráficos dinámicos.
- **Realidad Virtual y Gráficos 3D**
  - Three.js (Librería): Facilita la creación de gráficos 3D utilizando WebGL.
  - A-Frame (Framework): Permite construir experiencias de realidad virtual utilizando HTML y se basa en Three.js.
- **Geolocalización**
  - Geolocation API (API): Permite acceder a la ubicación geográfica del dispositivo, crucial para aplicaciones basadas en ubicación.
- **Peticiones asíncronas al servidor web**
  - Fetch (API): API nativa de JavaScript para realizar solicitudes HTTP.
  - Axios (Librería): Facilita la realización de solicitudes HTTP y manejo de respuestas con una API basada en promesas
- **Comunicación en Tiempo Real**

- WebSocket (API): Permite comunicación bidireccional y en tiempo real entre el cliente y el servidor.
- Socket.IO (Librería): Abstrae WebSockets y otras técnicas para proporcionar funcionalidades de tiempo real en todos los navegadores y dispositivos.

- **Trabajo en Segundo Plano**
  - WebWorker (API): Permite ejecutar procesos en el fondo sin afectar la interfaz de usuario.

El término API tiene múltiples definiciones, puedes ver un resumen de las principales en el siguiente enlace: https://aws.amazon.com/es/what-is/api/

Autoevaluación

¿Cuál de las siguientes tecnologías permite realizar peticiones asíncronas al servidor de forma más sencilla?

- [ ] Fetch.
- [ ] Three.js.
- [ ] WebSocket
- [ ] Axios

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Incorrecto
4. Correcto

2.5.- Testing en front-end

La validación (testing) es un componente esencial en el desarrollo de software. Un enfoque destacado en este ámbito es el desarrollo guiado por pruebas (t*est-driven development*), donde los test se diseñan antes incluso que el propio código.

Los *tests* permiten desde asegurar que una funcionalidad individual funciona correctamente hasta verificar la integración adecuada de múltiples componentes. Esto es especialmente crucial en proyectos colaborativos, donde es fundamental garantizar que los cambios en un elemento no impacten negativamente a otros, asegurando así la calidad de la aplicación.

Los principales tipos de test y las herramientas utilizadas para ejecutarlos en JavaScript son:

Tipo de Test	Descripción	Herramientas
Pruebas Unitarias	Evalúan la funcionalidad de un componente específico del software de manera aislada. Son útiles para asegurar que cada parte del código cumple con su funcionalidad designada.	✔ Jest: Especialmente popular en aplicaciones React y JavaScript en general. ✔ QUnit: Ideal para proyectos que utilizan jQuery. ✔ Mocha: Ofrece gran flexibilidad y es compatible con varios frameworks de aserciones.
Pruebas de Integración	Verifican cómo diferentes módulos o servicios trabajan juntos. Son clave para identificar problemas en las interfaces y la interacción entre componentes.	✔ Selenium WebDriver: Permite automatizar acciones en navegadores web para simular la interacción del usuario. ✔ Cypress: Una alternativa moderna a Selenium para pruebas más rápidas y en tiempo real.
Pruebas de Sistema	Involucran la validación completa del sistema para asegurar que el software cumple con los requisitos especificados.	✔ Visual Studio Profiler: Ayuda a identificar y solucionar cuellos de botella en el rendimiento de las aplicaciones. ✔ LoadRunner: Especializado en pruebas de rendimiento y carga, simulando miles de usuarios concurrentes.

Tipo de Test	Descripción	Herramientas
Pruebas de Aceptación	Se realizan para confirmar que el sistema está listo para el uso de los usuarios finales y cumple con sus necesidades.	✔ Cucumber: Facilita la escritura de pruebas en un lenguaje natural, permitiendo que los involucrados no técnicos participen en la definición de los comportamientos esperados.
Pruebas de Automatización	Utilizan herramientas para ejecutar pruebas repetitivas automáticamente, lo que ahorra tiempo y reduce la posibilidad de errores humanos. Especialmente útil cuando se incorporan nuevas funcionalidades.	✔ Selenium IDE: Una herramienta para grabar y reproducir interacciones en el navegador sin necesidad de escribir código.

Para saber más

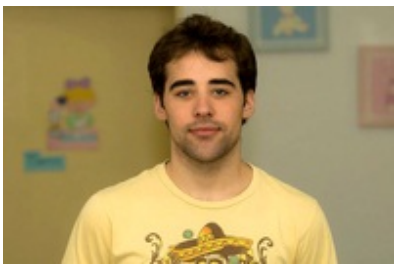
Algunos casos reales de fallos por una validación insuficiente son:

- ¡Café gratis! Agradécele al fallo informático en Starbucks.
- 7_fallos_de_software_por_falta_de_pruebas_que_conmocionaron_al_mundo (inglés).

Puedes ver más herramientas de testing en: https://en.wikipedia.org/wiki/List_of_unit_testing_frameworks#JavaScript

3.- Integración de código Javascript con HTML

Caso práctico



Ministerio de Educación (Uso educativo nc)

Antonio ha decidido ya el editor y los navegadores web que va a utilizar para programar con JavaScript, así que se pone manos a la obra y mira cuáles son los primeros pasos para integrar el nuevo código de JavaScript en el HTML.

Como **Antonio** ya conoce el lenguaje HTML, puesto que lo estudió en uno de los módulos que está cursando en su ciclo formativo, se centra en la parte específica de HTML que le permitirá integrar el nuevo lenguaje de programación JavaScript con el lenguaje HTML que ya conoce.

Ahora que ya conoces las herramientas que puedes utilizar para comenzar a programar en JavaScript, vamos a ver la forma de **integrar el código de JavaScript** en tu código HTML.

Los navegadores web te permiten varias opciones de inserción de código de JavaScript. Podremos insertar código usando las etiquetas `<script> </script>` y empleando un atributo `type` indicaremos qué tipo de lenguaje de script estamos utilizando:

Por ejemplo:

```
<script type="text/javascript">
      //  El código de JavaScript vendrá aquí.
</script>
```

Otra forma de integrar el código de JavaScript es utilizar un fichero externo que contenga el código de JavaScript y referenciar dicho fichero. Ésta sería la **forma más recomendable**, ya que así se consigue una separación entre el código y la estructura de la página web y como ventajas adicionales podrás compartir código entre diferentes páginas, centralizar el código para la depuración de errores, tendrás mayor claridad en tus desarrollos, más modularidad, seguridad del código y conseguirás que las páginas carguen más rápido. La rapidez de carga de las páginas se consigue al tener el código de JavaScript en un fichero independiente, ya que si más de una página tiene que acceder a ese fichero lo cogerá automáticamente de la caché del navegador con lo que se acelerará la carga de la página.

Para ello tendremos que añadir a la etiqueta `script` el atributo `src`, con el nombre del fichero que contiene el código de JavaScript. Generalmente los ficheros que contienen código JavaScript tendrán la extensión `.js`.

Por ejemplo:

```
<script  type="text/javascript"  src="miScript.js"></script>
```

Si necesitas cargar más de un fichero **.js** repite la misma instrucción cambiando el nombre del fichero. Las etiquetas de **<script>** y **</script>** son obligatorias a la hora de incluir el fichero **.js**. **Atención**: no escribas ningún código de JavaScript entre esas etiquetas cuando uses el atributo **src** .

Para **referenciar el fichero origen** **.js** de JavaScript dependerá de la localización física de ese fichero. Por ejemplo en la línea anterior el fichero **miScript.js** deberá estar en el mismo directorio que el fichero **.html**. Podrás enlazar fácilmente a otros ficheros de JavaScript localizados en directorios diferentes de tu servidor o de tu dominio. Si quieres hacer una referencia absoluta al fichero, la ruta tendrá que comenzar por **http://**, en otro caso tendrás que poner la ruta relativa dónde se encuentra tu fichero **.js**.

Ejemplos:

```
<script  type="text/javascript"  src="http://www.midominio.com/miScript.js"></script>
```

En este caso estamos referenciando nuestro script mediante una referencia absoluta, que hace referencia a nuestro dominio en primer lugar y posteriormente a la ruta dentro de nuestro domino. Es decir, estamos referenciando mediante una URL que se refiere a nuestro domino.

```
<script  type="text/javascript"  src="./js/miScript.js"></script>
```

En este caso estamos referenciando nuestro script mediante una referencia relativa, que hace referencia a nuestro script desde el mismo directorio o carpeta en la que se encuentra la página **.html** en la que está incrustado dicho código, pero situado en el directorio **js**.

Cuando alguien examine el código fuente de tu página web verá el enlace a tu fichero **.js**, en lugar de ver el código de JavaScript directamente. Esto no quiere decir que tu código no sea inaccesible, ya que simplemente copiando la ruta de tu fichero **.js** y tecleándola en el navegador podremos descargar el fichero **.js** y ver todo el código de JavaScript. En otras palabras, nada de lo que tu navegador descargue para mostrar la página web podrá estar oculto de la vista de cualquier programador.

A veces te puedes encontrar que tu script se va a ejecutar en un navegador que no soporta JavaScript. Para ello dispones de una etiqueta **<noscript>Texto informativo</noscript>** que te permitirá indicar un texto adicional que se mostrará indicando que ese navegador no soporta JavaScript. Pero eso no es lo habitual hoy día.

Debes conocer

Para asegurar que un script se ejecute cuando el DOM ha cargado, puedes añadir el atributo **defer**. Por otro lado, para que un script se ejecute tan pronto como esté listo, se utiliza el atributo **async**. Ambos atributos solo afectan a scripts externos (aquellos importados con src) y no tienen efecto en scripts definidos directamente en el archivo HTML.

```
<script  type="text/javascript"  src="./js/miScript.js" defer></script>
```

Puedes profundizar más sobre estos atributos en: https://es.javascript.info/script-async-defer

Autoevaluación

Al utilizar un fichero externo de JavaScript referenciado mediante el atributo `src`, no se debe incluir ningún código JavaScript dentro de las etiquetas `<script></script>` en el archivo HTML

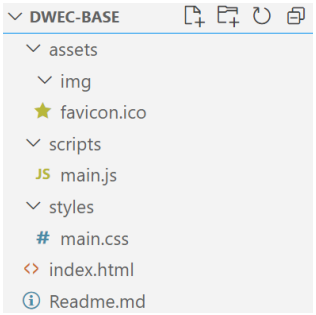
○ Verdadero ○ Falso

Verdadero

Verdadero. Al utilizar el atributo `src` para referenciar un fichero externo de JavaScript, no se debe escribir código JavaScript dentro de las etiquetas `<script></script>`. Estas etiquetas deben estar vacías, limitándose a incluir el atributo `src` con la ruta al fichero.

3.1.- Estructura de carpetas

La organización eficiente de las carpetas en un proyecto es crucial para mantener los archivos ordenados por tipo y propósito, facilitando así su escalabilidad y colaboración entre múltiples desarrolladores. Dependiendo de la tecnología empleada, la estructura de carpetas puede variar significativamente. Por ejemplo, un proyecto que utiliza únicamente JavaScript se organiza de manera distinta a uno que incorpora frameworks como React.



Óscar Gómez. *Sistema de carpetas de un proyecto Web simple*. (Dominio público)

En un proyecto JavaScript típico, la estructura podría organizarse de la siguiente manera:

- **./assets/:** Esta carpeta almacena los recursos visuales y gráficos del proyecto, como imágenes, iconos y fuentes tipográficas.
- **./scripts/** (o alternativamente **./js/** en proyectos JavaScript y **./ts/** en proyectos TypeScript): Aquí se guardan los scripts, generalmente organizados por categorías según su funcionalidad.
- **./styles/** (o en su variante **./css/**): Se destinan para los archivos de estilos, ya sean en formato .css o archivos .sass que serán compilados a CSS.

Para saber más

Puedes ver otras propuestas de organización de carpetas en los siguientes artículos:

- https://developer.mozilla.org/es/docs/Learn/Getting_started_with_the_web/Dealing_with_files
- https://medium.com/@orlidev/estructuras-de-carpetas-en-proyectos-web-una-gu%C3%ADa-al-estilo-de-los-sims-1ceb4cf9fdd8

3.2.- HolaMundo.js

Para insertar código Javascript en HTML (suponemos que trabajamos en HTML5) utilizaremos el siguiente código:

- Insertando el código directamente en el archivo **index.html**:

```
<!DOCTYPE html>
<html lang="es-ES">
    <head>
        <meta charset="UTF-8">
        <title>Hola mundo desde DWEC</title>
        <script>
            const saludar = () => {
                alert('Hola mundo desde DWEC');
            }
        </script>
    </head>
    <body>
        <h1>
            Hola Mundo desde DWEC
        </h1>
        <button onclick="saludar()">Saludar</button>
    </body>
</html>
```

- Haciendo referencia al código existente en otro archivo **.js**:
  Creamos un archivo nombrado **miScript.js** en la carpeta *scripts* cuyo contenido es el siguiente:

```
const saludar = () => {
    alert('Hola mundo desde DWEC');
}
```

  Y ahora lo referenciamos desde el archivo **index.html**:

```
<!DOCTYPE html>
<html lang="es-ES">
    <head>
        <meta charset="UTF-8">
        <title>Hola mundo desde DWEC</title>
        <script src="./scripts/miScript.js"></script>
    </head>
    <body>
        <h1>
            Hola Mundo desde DWEC
        </h1>
        <button onclick="saludar()">Saludar</button>
```

```
    </body>
</html>
```

Para saber más

JavaScript se ejecuta por defecto en modo poco riguroso (*sloppy mode*) permitiendo errores tales como asignar un valor a una variable no declarada. Para evitar posibles errores silenciosos se recomienda utilizar el modo estricto, para ello basta con poner al principio de nuestro código JavaScript la siguiente línea:

```
"use strict";
```

Autoevaluación

¿Es necesario incluir el atributo `type="text/javascript"` en la etiqueta `<script>` al referenciar un archivo JavaScript externo en HTML5?

○ Verdadero ○ Falso

Verdadero

Falso. En HTML5, no es necesario incluir el atributo `type="text/javascript"` en la etiqueta `<script>` porque el tipo de script se asume automáticamente como JavaScript.

3.3.- Ofuscación de código

Seguramente estarás pensando en cómo puedes **proteger el código de JavaScript** que vas a programar, del uso fraudulento por otros programadores o visitantes a tu página: la respuesta rápida a esa pregunta es que es imposible hacerlo.

Para que el código de JavaScript pueda ejecutarse correctamente deberá ser cargado por el navegador web y por lo tanto su código fuente deberá estar visible al navegador. Si realmente te preocupa que otras personas usen o roben tus scripts, deberías incluir un mensaje de copyright en tu código fuente. Piensa que no solamente tus scripts son visibles al mundo, sino que los scripts del resto de programadores también lo son. De esta forma puedes ver fácilmente cuando alguien está utilizando al pie de la letra tus scripts, aunque esto no evita que alguien copie tu código y elimine tu mensaje de copyright.

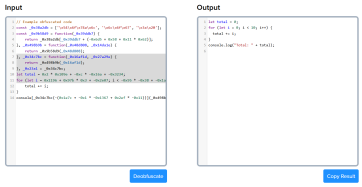
Lo más que se puede hacer es ofuscar el código, o hacerlo más difícil de entender. Las técnicas de ofuscación incluyen la eliminación de saltos de línea, espacios en blanco innecesarios, tabuladores, utilización de nombres ininteligibles en las funciones y variables, utilización de variables para almacenar trozos de código, uso de recursividad, etc. La forma más rápida de hacer todas esas tareas de ofuscación es utilizar un software que producirá una copia comprimida del script que has programado para facilitar su carga rápida.

Para que veas un ejemplo de ofuscador de JavaScript visita:

Herramienta para ofuscar JavaScript.

Lo mejor es que cambies ese paradigma y pienses de una manera diferente. En lugar de proteger tu código, lo mejor es promocionarlo y hacer ostentación de él, añadiendo comentarios útiles en el código fuente, publicándolo en tu blog o en webs de programación, etc. Lo suyo sería que lo compartieras con toda la comunidad, con la licencia que estimes oportuno, por ejemplo, en GitHub.

Debes conocer

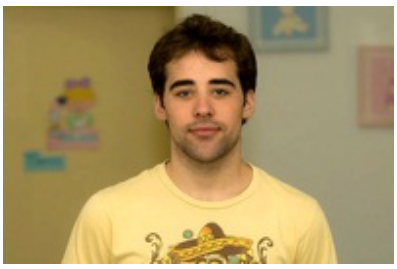


Óscar Gómez. *Desofuscar código con JavaScript Deobfuscator*. (Dominio público)

También existen herramientas de desofuscación de código. Aunque estas no permiten recuperar exactamente el código original, especialmente en aspectos como comentarios o nombres de variables, sí facilitan la obtención de un código mucho más legible.

4.- Herramientas y utilidades de programación

Caso práctico



Ministerio de Educación (Uso educativo nc)

En **BK Programación Juan** y **Antonio** han decidido, después de analizar los requisitos del proyecto y de consultarlo con su directora **Ada**, que utilizarán el lenguaje JavaScript para la realización del proyecto de modernización de la empresa de moda.

Lo primero que tienen que hacer es decidir qué tipo de herramientas y utilidades adicionales van a usar para realizar la programación con el lenguaje JavaScript. Buscan alguna herramienta que les permita introducir el código de JavaScript fácilmente y les aporte ayudas adicionales detectando errores sintácticos, partes incompletas, etc.

También tienen que decidir qué navegador o navegadores van a usar para comprobar la ejecución y compatibilidad de su código de JavaScript.

Aunque es posible programar una página web usando únicamente un simple editor de texto, existen múltiples herramientas y utilidades que facilitan significativamente el desarrollo de un proyecto. Estas herramientas permiten agilizar el proceso, minimizar errores y mejorar la eficiencia general. En esta sección, repasaremos las herramientas más importantes, desde las herramientas de desarrollo integradas en los navegadores web o los entornos de desarrollo integrado (IDE) hasta el control de versiones con plataformas como GitHub.

4.1.- Herramientas del navegador web

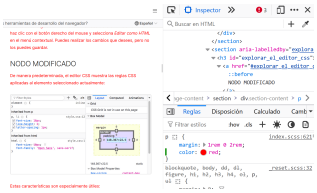
Los navegadores web tienen una serie de herramientas diseñadas para los desarrolladores (puedes acceder a ellas simplemente pulsando **F12**). Incluso existen algunos navegadores webs diseñados específicamente para desarrolladores webs como: Firefox Developer y Chrome Dev.

Sus principales herramientas son:

Explorando el DOM

Una de las principales utilidades del DOM (Modelo de Objetos del Documento) es poder inspeccionar el código HTML de cualquier elemento en un sitio web, así como modificar sus atributos, estilo y contenido.

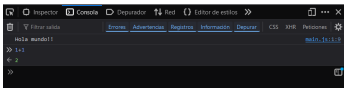
En el desarrollo web del lado cliente, esta capacidad resulta especialmente útil para consultar el ID y la clase de los nodos que se desean manipular con JavaScript.



Óscar Gómez. *Herramientas del desarrollador. Inspector web (firefox).* (Dominio público)

Consola

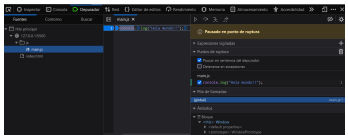
La consola permite ejecutar código JavaScript, además de mostrar registros (*log*) de mensajes, avisos y errores del sistema.



Óscar Gómez. *Herramientas del desarrollador. Consola.* (Dominio público)

Depurador

El depurador del navegador facilita establecer puntos de ruptura, seguir el flujo de ejecución del código y observar el estado de las variables en tiempo real.



Óscar Gómez. *Herramientas del desarrollador. Depurador.* (Dominio público)

Es una herramienta imprescindible para detectar errores y validar el comportamiento adecuado de una aplicación JavaScript.

Y más

Además, existen muchas más herramientas útiles para desarrolladores como:

- **Almacenamiento**: Permite inspeccionar la información que las webs almacenan localmente en el navegador.
- **Red**: Facilita la revisión y análisis de los paquetes de datos enviados y recibidos durante la navegación.
- **Accesibilidad**: Esencial para comprobar la conformidad con los estándares de accesibilidad, incluyendo aspectos como el contraste de colores y la adecuación de las descripciones de imágenes.
- ***Responsive***: Ofrece la posibilidad de evaluar cómo se visualiza una página web en distintos dispositivos y tamaños de pantalla, asegurando una experiencia óptima para todos los usuarios.

Para saber más

En los siguientes enlaces puedes ver tutoriales de uso de las herramientas de desarrollo web:

- Firefox: https://developer.mozilla.org/es/docs/Learn/Common_questions/Tools_and_setup/What_are_browser_developer_tools
- Chrome: https://medium.com/@diego.coder/un-vistazo-a-las-chrome-devtools-38e780502932
- Edge: https://learn.microsoft.com/es-es/microsoft-edge/devtools-guide-

chromium/overview

Autoevaluación

La herramienta de inspección del DOM permite modificar el código HTML de cualquier elemento en un sitio web directamente desde el navegador

○ Verdadero ○ Falso

Verdadero

La herramienta de inspección del DOM permite a los desarrolladores inspeccionar y modificar el código HTML de cualquier elemento en un sitio web directamente desde el navegador. Estos cambios son solamente locales y se perderán al recargar la página.

4.2.- Editor de texto e IDEs

La elección del editor depende de ti, pero aquí te voy a dar algunas pistas para realizar una buena elección.

Para aprender JavaScript no se recomiendan editores del estilo WYSIWYG, ya que estas herramientas están más orientadas a la modificación de contenido y presentación, y nosotros nos vamos a centrar más en el código fuente de la página.

Uno de los **factores** importantes que tienes que tener en cuenta a la hora de elegir un editor, es ver la facilidad con la que se pueden grabar los ficheros con extensión **.html**. Independientemente del sistema operativo que estés utilizando, cualquier programa que te permita grabar ficheros directamente con dicha extensión te evitaría un gran número de problemas. También hay que tener en cuenta la codificación que emplea ese programa para grabar los ficheros.

Hoy día no es necesario contar con un IDE para desarrollar en Javascript. Hay editores ligeros que, a día de hoy, nos ofrecen casi las mismas características. Lo que sí debes tener en cuenta es que te aporten las siguientes características para hacer tu trabajo más fácil:

- **Resaltado de texto.** Muestra con distinto color o tipo de letra los diferentes elementos del lenguaje: sentencias, variables, comentarios, etc. También genera indentado automático para diferenciar de forma clara los distintos bloques de un programa.
- **Completado automático.** Detecta qué estás escribiendo y cuando es posible te muestra distintas opciones para completar el texto.
- **Navegación en el código.** Permite buscar de forma sencilla elementos dentro del texto, por ejemplo, definiciones de variables.
- **Comprobación de errores al editar.** Reconoce la sintaxis del lenguaje y revisa el código en busca de errores mientras lo escribes.
- **Generación automática de código.** Ciertas estructuras, como la que se utiliza para las clases, se repiten varias veces en un programa. La generación automática de código puede encargarse de crear la estructura básica, para que sólo tengas que rellenarla.
- **Gestión de versiones.** En conjunción con un sistema de control de versiones, el entorno de desarrollo te puede ayudar a guardar copias del estado del proyecto a lo largo del tiempo, para que si es necesario puedas revertir los cambios realizados.

Editores más populares para trabajar con javascript

VSCode

Visual Studio Code de Microsoft es una herramienta muy versátil, a la que se le pueden instalar muchas extensiones para darle una gran funcionalidad en casi cualquier lenguaje de programación. Visual Studio



Microsoft Visual Code (Todos los derechos reservados)

Code.

Atom

Atom de GitHub es una herramienta también muy ligera y que también cuenta con multitud de extensiones para adaptarla a nuestras necesidades. Atom.



Atom (Todos los derechos reservados)

SublimeText

SublimeText al igual que en los anteriores se le pueden instalar gran cantidad de extensiones. Aunque es software propietario, se puede usar para enseñanza.



Sublime Text (Todos los derechos reservados)

Otro de los componentes obligatorio para aprender JavaScript es el navegador web. No es necesario que te conectes a Internet para comprobar tus scripts realizados con JavaScript. Puedes realizar dicha tarea sin conexión. Esto quiere decir que puedes aprender JavaScript y crear aplicaciones con un ordenador portátil y desde cualquier lugar sin necesitar Internet para ello.

Lo que sí es importante es utilizar las extensiones que nos ofrecen muchos de los navegadores ya que te van a permitir ver las salidas por consola que anuncian errores, ver la estructura del documento, incluso en algunos casos depurarlos, etc.

4.2.1. Visual Studio Code

Visual Studio Code es un editor de código fuente desarrollado por Microsoft, diseñado para proporcionar una experiencia de desarrollo fluida y eficiente. Con una amplia gama de características como depuración integrada, control de versiones, snippets, y una extensibilidad robusta mediante extensiones, VS Code se ha convertido en una herramienta esencial para desarrolladores web de todos los niveles.

> Es recomendable echarle un vistazo a la documentación oficial, y en particular a su sección dedicada a JavaScript, para poder sacarle todo el partido posible a Visual Studio Code.

Algunas de las principales características y extensiones de Visual Studio Code son:

- **Ejecución** del archivo index.html en un servidor con Live Server. La web se recarga ante cualquier cambio que hagamos en el código.
- **Autocompletado y Snippets** de JavaScript con IntelliCode y JavaScript (ES6) code snippets.
- Los **snippets** son fragmentos de código predefinidos que facilitan la escritura de código repetitivo. Se pueden activar con **CTRL + Espacio**. Por ejemplo, con HTML Boilerplate, si se escribe HTML en un documento y se pulsa **CTRL + Espacio** se genera el siguiente fragmento de código:

```
<!DOCTYPE html>
<!--[if lt IE 7]>      <html class="no-js lt-ie9 lt-ie8 lt-ie7"> <![endif]-->
<!--[if IE 7]>         <html class="no-js lt-ie9 lt-ie8"> <![endif]-->
<!--[if IE 8]>         <html class="no-js lt-ie9"> <![endif]-->
<!--[if gt IE 8]>      <html class="no-js"> <!--<![endif]-->
<html>
    <head>
        <meta charset="utf-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <title></title>
        <meta name="description" content="">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <link rel="stylesheet" href="">
    </head>
    <body>
        <!--[if lt IE 7]>
            <p class="browsehappy">You are using an <strong>outdated</strong> browser. Please <
        <![endif]-->

        <script src="" async defer></script>
    </body>
</html>
```

- **Hacer uso de las utilidades para Debug**: VS Code permite depurar aplicaciones con facilidad. A continuación, se muestra un ejemplo de archivo de configuración para *debug* (**launch.json**):

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Launch Chrome against index.html",
            "type": "chrome",
            "request": "launch",
            "file": "${workspaceFolder}/index.html",
        },
        {
            "name": "Launch Chrome against localhost",
            "type": "chrome",
            "request": "launch",
            "url": "http://localhost:8080",
            "webRoot": "${workspaceFolder}"
        }
    ]
}
```

- **Detección de errores** con EsLint. Esta extensión requiere configuración, veremos cómo hacerlo en el apartado 4.5.
- Coloreado del código HTML asignado a una variable con es6-string-html.
- Probar **peticiones AJAX** a un servidor con Thunder Client.
- **Atajos de teclado** como:
  - Comentado y descomentado del código con el atajo **CTRL + Ç**.
  - Formateado automático del código con Pretier con el atajo **CTRL + SHIFT + P**.
  - Ajustar las líneas de código a la ventana del editor (wrap-lines) con el atajo **ALT + Z**.
  - Consultar los atajos de teclado más comunes es altamente recomendable.
- Y un lago etcétera, con extensiones para integración con git, el uso del terminal directamente en visual studio code o extensiones específicas para los diferentes frameworks.

Para saber más

Para obtener más información, se recomienda visitar la sección de tips y trucos en la documentación oficiall.

Lecturas adicionales recomendadas:

- Guía de las 20 mejores extensiones de VSCode para desarrolladores

frontend.

- Configuraciones y extensiones de VSCode para un desarrollo más rápido en JavaScript.
- Top extensiones de VSCode para un desarrollo más rápido.

Con estos recursos y configuraciones, podrás sacar el máximo provecho de Visual Studio Code en tus proyectos de desarrollo web.

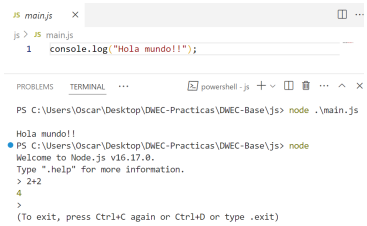
4.3.- Node.js

Node.js® es un entorno de ejecución JavaScript gratuito, de código abierto y multiplataforma que permite a los desarrolladores crear servidores, aplicaciones web, herramientas de línea de comandos y scripts. Se utiliza habitualmente para crear servidores back-end utilizando código javascript en combinación con librerías tales como Express o Mongoose.



Node.js. (Todos los derechos reservados)

Una ventaja de tener Node.js en nuestro equipo es que no permite abrir un intérprete de JavaScript en el terminal, así como ejecutar scripts `.js` en el terminal (siempre que no requieran el acceso a elementos del archivo `html`, es decir, del DOM).



Óscar Gómez. *Ejemplo uso Node.js.* (Dominio público)

Autoevaluación

Node.js permite ejecutar código JavaScript en el lado del servidor, fuera del navegador web

○ Verdadero ○ Falso

Verdadero

Verdadero. Node.js es un entorno de ejecución que permite ejecutar código JavaScript en el lado del servidor, fuera del navegador.

4.4.- Gestión paquetes con NPM

NPM es un gestor de paquetes para proyectos JavaScript, tanto de front-end como de back-end. Permite instalar librerías (como EsLint para comprobar errores o Axios para hacer peticiones al servidor) en nuestro proyecto que son almacenadas en la carpeta *node_modules*.



NPM. *Logo NPM.* (Todos los derechos reservados)

Con NPM instalado, se puede inicializar un proyecto ejecutando **npm init** en el terminal en la raiz del proyecto. El terminal irá pidiéndonos información del proyecto, como nombre del autor o el repositorio git, con el que crea un archivo *package.json* con la información del proyecto. Como el siguiente:

```
{
  "name": "dwec-u1-holamundojs_2",
  "version": "1.0.0",
  "description": "",
  "main": "main.js",
  "browserslist": [ "last 2 years"],
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Oscar Gomez",
  "license": "ISC",
  "dependencies": {
    "axios": "^1.6.8"
  },
  "devDependencies": {
    "eslint": "^8.24.0",
    "eslint-plugin-compat": "^4.0.2"
  }
}
```

Para instalar un paquete en nuestro proyecto basta con ejecutar el comando: **npm install <nombre_paquete>**. Por ejemplo, para instalar axios bastaría con ejecutar: **npm install axios**.

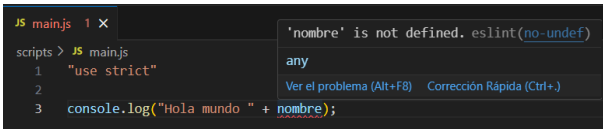
Para compartir un proyecto se debe omitir (o borrar) la carpeta *node_modules*, debido a su gran tamaño y numeroso número de ficheros. Para restaurarla en el nuevo equipo basta con ejecutar: **npm init** y **npm install**.

Para saber más

Existen otros gestores de paquetes para proyectos Javascript como Yarn.

4.5.- Detección de errores con EsLint

Eslint es una herramienta que analiza el código fuente para detectar patrones problemáticos (p. ej. una variable no declarada) que podrían causar problemas a la hora de interpretar o compilar el código.

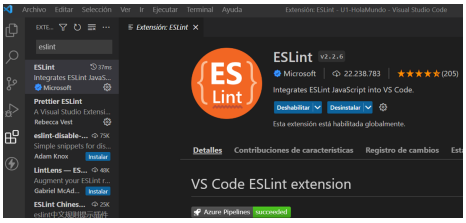


Óscar Gómez. *Ejemplo de uso Eslint (1): Variable no declarada.* (Dominio público)

Puesta en marcha de EsLint

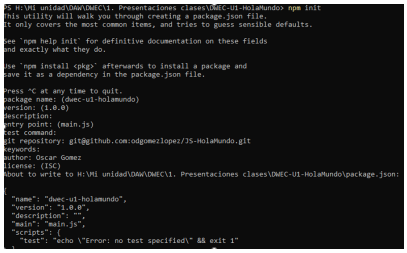
Para poner en funcionamiento EsLint en un proyecto JavaScript en Visual Studio Code hay que seguir los siguientes pasos:

- Instalar la extensión de EsLint en Visual Studio Code.



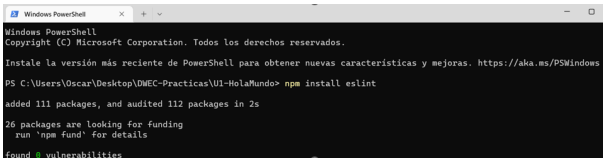
Óscar Gómez. *Instalar la extensión de EsLInt para Visual Studio Code.* (Dominio público)

- Inicializar el proyecto con: **npm init**



Óscar Gómez. *Inicializar proyecto con npm init.* (Dominio público)

- Instalar el paquete EsLint con: **npm install -g eslint**

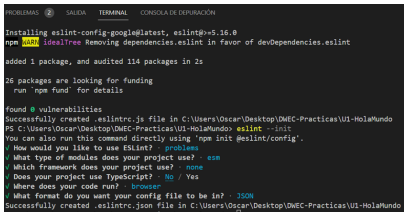


Óscar Gómez. *Installar EsLint con NPM.* (Dominio público)

Si el terminal no te encuentra el comando **npm** puedes solucionarlo siguiendo el

siguiente tutorial: https://bertofern.wordpress.com/2019/01/08/solucion-node-js-npm-no-reconocido-como-comando-interno-o-externo/

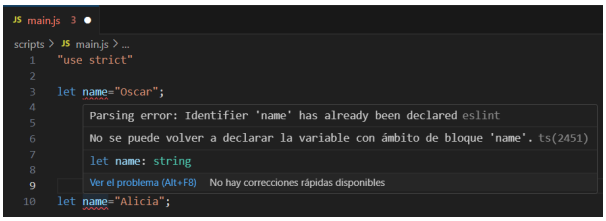
- Configurar eslint para nuestro proyecto actual ejecutando: **eslint --init**. Se debe responder a una serie de preguntas indicando las tecnologías que utilizamos para el proyecto actual.



Óscar Gómez. *Inicializar y configurar EsLint.* (Dominio público)

El último comando puede fallar si PowerShell no tiene permiso para ejecutar scripts. Para solucionarlo basta con cambiar la Execution Policy. Por ejemplo, ejecutando con permisos de administrador el siguiente comando: **set-executionpolicy remotesigned**

Una vez realizado este último paso, EsLint ya está en funcionamiento y permite detectar errores y avisos en nuestro código.



Óscar Gómez. *Ejemplo de uso Eslint (2): Redeclaración de variable.* (Dominio público)

Recomendación

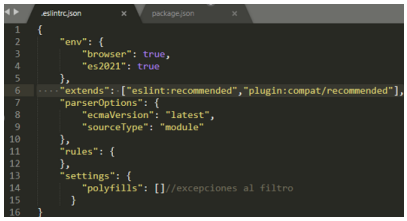
Para no tener que repetir este proceso en cada proyecto, se recomienda tener un proyecto con la configuración básica y utilizarlo como plantilla. Esto se puede hacer guardado una copia en nuestro equipo o creando una plantilla con nuestro proyecto en github.

Para saber más

EsLint cuenta con diversos *plugins* para extender sus funcionalidades. Uno de los más utilizados es eslint-plugin-compat. Este plugin permite detectar instrucciones JavaScript que son ejecutables en un navegador, pero no en otros. Esto se puede consultar manualmente en la web de https://caniuse.com/.

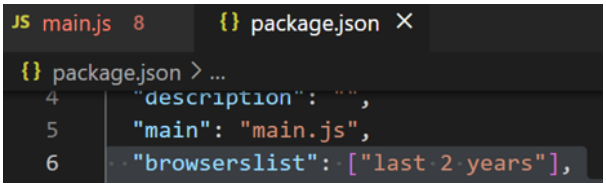
Puesta en marcha:

- Instalar el *plugin* ejecutando: **npm install --save-dev eslint-plugin-compat**
- Modificar el archivo de configuración de .eslintrc.json (ó .js) añadiendo la siguiente línea: **"extends": ["eslint:recommended","plugin:compat/recommended"],**



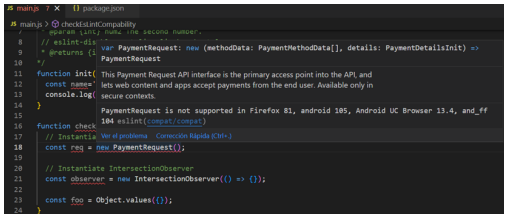
Óscar Gómez. *EsLint. Detectar problemas de compatibiidad entre navegadores.* (Dominio público)

- En package.json, se puede modificar la lista de navegadores con los que se quiere revisar la compatibilidad.



Óscar Gómez. *Eslint. Lista de navegadores para la comprobación de la compatibilidad.* (Dominio público)

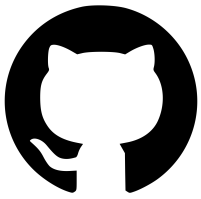
Y ya está en funcionamiento (a veces puede ser necesario recargar el navegador, por ejemplo, pulsando **F1** y ejecutando la acción **Reload Window**):



Óscar Gómez. *EsLint. Ejemplo de problemas de compatibilidad.* (Dominio público)

4.6.- Gestión de versiones con Github

GitHub es una poderosa herramienta de control de versiones que transforma la manera en que gestionamos nuestros proyectos. Permite llevar un registro detallado de todos los cambios realizados, brindándonos la flexibilidad de retroceder en el tiempo si es necesario. Este aspecto es invaluable para mantener la integridad y la continuidad de nuestros proyectos.



GitHub. (Todos los derechos reservados)

Además, GitHub está diseñado para el trabajo colaborativo. Facilita la creación de múltiples ramas en las que los equipos pueden trabajar en paralelo, permitiendo que cada miembro aporte sus cambios sin interferir con el trabajo de los demás. Eventualmente, estas ramas se integran en la rama principal a través de merges, asegurando que todas las contribuciones se unan de manera ordenada y eficiente.

Debes conocer

- Tutorial para crear una cuenta de Github y crear el primer proyecto: https://www.freecodecamp.org/espanol/news/guia-para-principiantes-de-git-y-github/
- Archivo .gitignore para proyectos web (elegible al crear el proyecto bajo el nombre de Node), permite ignorar las dependencias del proyecto instaladas con NPM (la carpeta `node_modules`, recuerda que para restaurarla bastará con ejecutar: **npm init** y **npm install**): https://github.com/odgomezlopez/DWEC-U1-JS-HolaMundo/blob/main/.gitignore
- Tutorial de uso de Github Desktop (interfaz gráfica para el uso de github): https://docs.github.com/es/desktop/overview/creating-your-first-repository-using-github-desktop

Otro uso importante de GitHub es la construcción de un portafolio de proyectos. Esta plataforma permite mostrar tu trabajo y habilidades de manera profesional, lo que puede hacer tu currículum vitae más atractivo para potenciales empleadores. Tener un portafolio en GitHub demuestra tus capacidades y tu experiencia práctica, añadiendo un valor significativo a tu perfil profesional.

Recomendación

Se recomienda tener un proyecto plantilla en GitHub con la configuración básica que necesitas. Esto te permite clonar esta plantilla para nuevos proyectos, evitando la necesidad de rehacer la configuración básica cada vez.

Puedes ver un ejemplo en el siguiente repositorio: https://github.com/odgomezlopez/DWEC-U1-JS-HolaMundo

Para saber más

GitHub Pages te permite crear un sitio web de solo front-end para mostrar tus proyectos.

Autoevaluación

La carpeta `node_modules` debe incluirse en el control de versiones en GitHub para asegurar que todos los módulos y dependencias estén disponibles para otros colaboradores del proyecto

○ Verdadero ○ Falso

Falso

Falso. La carpeta `node_modules` generalmente no se incluye en el control de versiones en GitHub. En lugar de ello, se utiliza un archivo `.gitignore` para excluir esta carpeta. Las dependencias se definen en el archivo `package.json`, y los colaboradores pueden reinstalarlas fácilmente usando el comando `npm install`.